

```
In [3]: import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
```

Data collection and preprocessing

```
In [4]: data = pd.read_csv(r"loanprediction.csv")
data.head()
```

```
Out[4]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	1
1	LP001003	Male	Yes	1	Graduate	No	4583	1
2	LP001005	Male	Yes	0	Graduate	Yes	3000	1
3	LP001006	Male	Yes	0	Not Graduate	No	2583	1
4	LP001008	Male	No	0	Graduate	No	6000	1

```
In [5]: data.shape
```

```
Out[5]: (614, 13)
```

```
In [6]: data.describe()
```

```
Out[6]:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842182
std	6109.041673	2926.248369	85.587325	65.120411	0.364818
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

```
In [7]: data.isna().sum()
```

```
Out[7]: Loan_ID      0
        Gender      13
        Married      3
        Dependents  15
        Education    0
        Self_Employed 32
        ApplicantIncome 0
        CoapplicantIncome 0
        LoanAmount    22
        Loan_Amount_Term 14
        Credit_History 50
        Property_Area  0
        Loan_Status    0
        dtype: int64
```

```
In [8]: # dropping the missing values
        data = data.dropna()
```

```
In [9]: data.isna().sum()
```

```
Out[9]: Loan_ID      0
        Gender      0
        Married      0
        Dependents    0
        Education     0
        Self_Employed 0
        ApplicantIncome 0
        CoapplicantIncome 0
        LoanAmount     0
        Loan_Amount_Term 0
        Credit_History 0
        Property_Area  0
        Loan_Status    0
        dtype: int64
```

```
In [10]: data.shape
```

```
Out[10]: (480, 13)
```

```
In [11]: data.columns
```

```
Out[11]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
                'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
                'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
                dtype='str')
```

```
In [12]: data.Loan_Status.value_counts()
```

```
Out[12]: Loan_Status
Y      332
N      148
Name: count, dtype: int64
```

```
In [13]: # Label encoding
        enc_status= {'N':0, 'Y':1}
```

```
data.Loan_Status = data.Loan_Status.map(enc_status)
```

```
In [14]: data.Loan_Status.value_counts()
```

```
Out[14]: Loan_Status  
1      332  
0      148  
Name: count, dtype: int64
```

```
In [15]: # Dependent column values  
data.Dependents.value_counts()
```

```
Out[15]: Dependents  
0      274  
2       85  
1       80  
3+       41  
Name: count, dtype: int64
```

```
In [16]: # replace +3 values to 4  
data = data.replace(to_replace='3+', value=4)
```

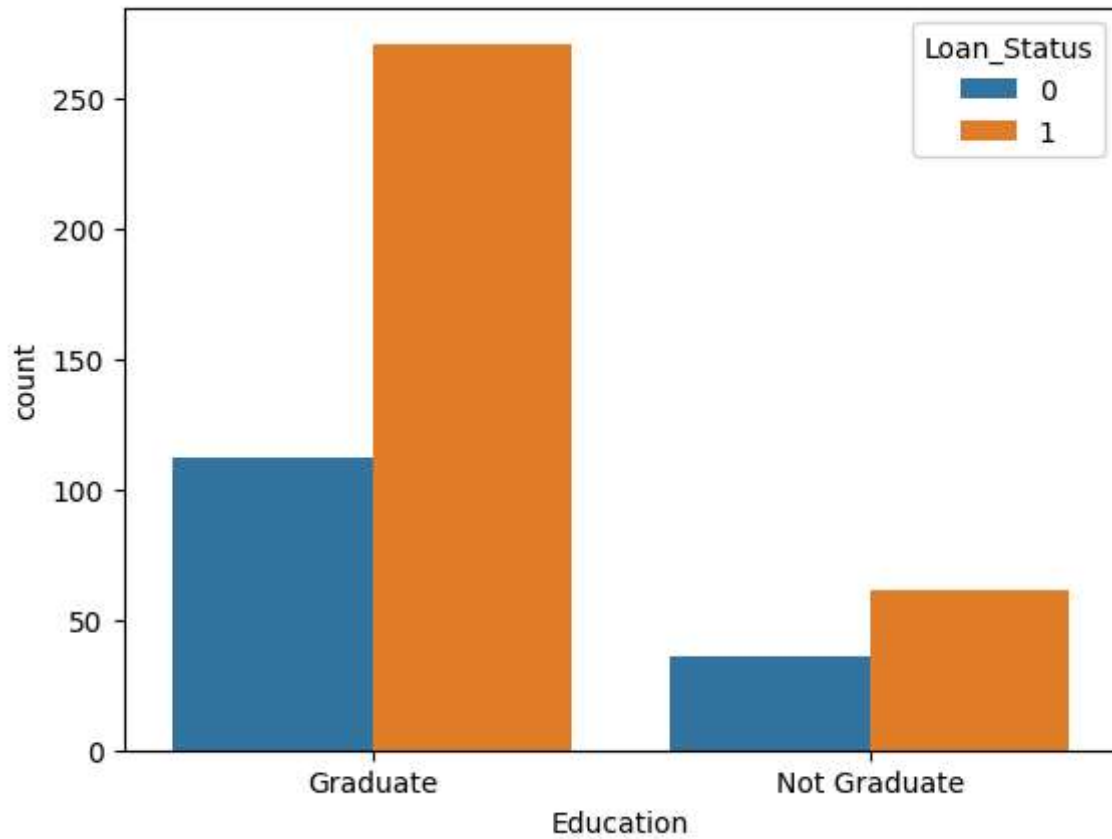
```
In [17]: data.Dependents.value_counts()
```

```
Out[17]: Dependents  
0      274  
2       85  
1       80  
4       41  
Name: count, dtype: int64
```

Data Visualization

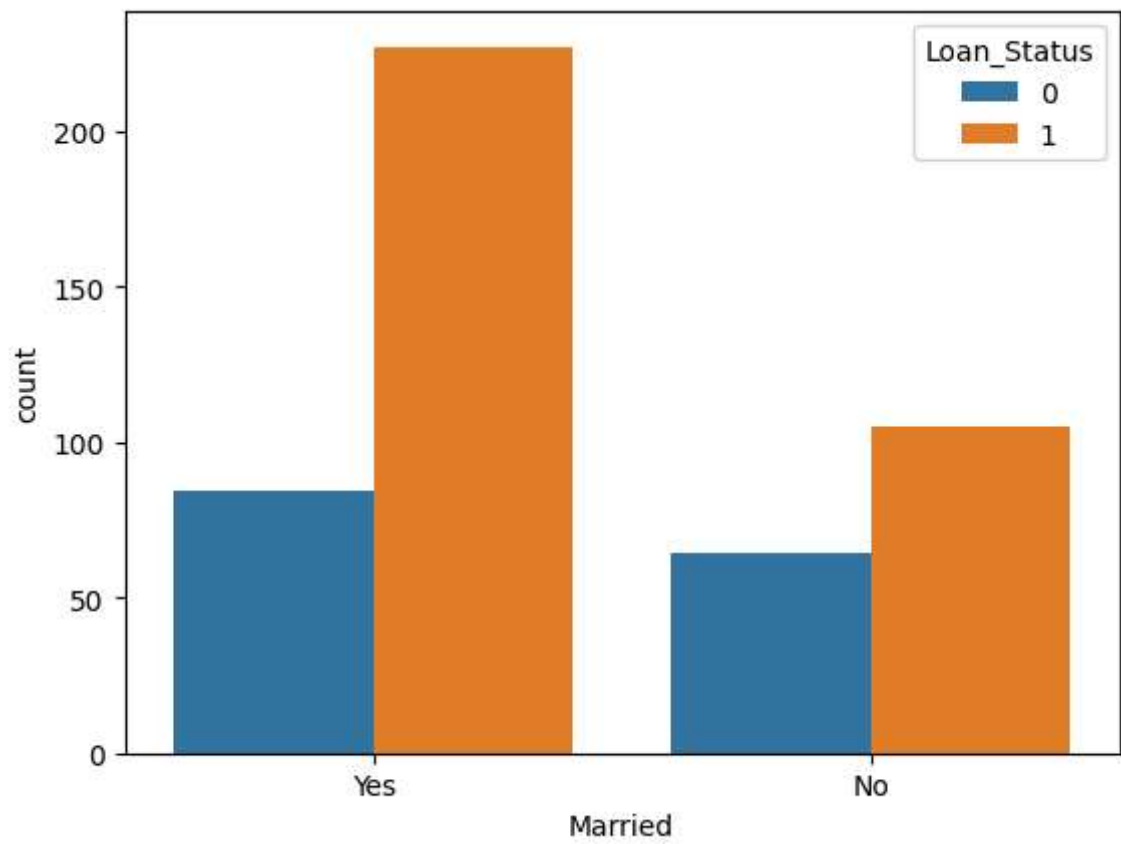
```
In [18]: # education and loan status  
sns.countplot(x='Education', hue='Loan_Status', data=data)
```

```
Out[18]: <Axes: xlabel='Education', ylabel='count'>
```



```
In [19]: # marital status and loan status  
sns.countplot(x='Married', hue='Loan_Status', data=data)
```

```
Out[19]: <Axes: xlabel='Married', ylabel='count'>
```



```
In [20]: # convert categorical columns to numerical values
data.replace({'Married': {'No':0, 'Yes':1}, 'Gender':{'Male':1, 'Female':0}, 'Self_Empl': {'No':0, 'Yes':1}, 'Property_Area': {'Rural':0, 'Semiurban':1, 'Urban':2}, 'Education':{'Grad':0, 'PostGrad':1, 'Diploma':2, 'HighSchool':3}}
```

Out[20]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
1	LP001003	1	1	1	1	0	4583
2	LP001005	1	1	0	1	1	3000
3	LP001006	1	1	0	0	0	2583
4	LP001008	1	0	0	1	0	6000
5	LP001011	1	1	2	1	1	5417
...	...	...	...	...	...	...	...
609	LP002978	0	0	0	1	0	2900
610	LP002979	1	1	4	1	0	4106
611	LP002983	1	1	1	1	0	8072
612	LP002984	1	1	2	1	0	7583
613	LP002990	0	0	0	1	1	4583

480 rows × 13 columns



```
In [21]: # seperating the data and label
X = data.drop(columns=['Loan_ID', 'Loan_Status'])
y = data['Loan_Status']
```

```
In [22]: X.head()
```

```
Out[22]:
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplicant
1	1	1	1	1	0	4583	
2	1	1	0	1	1	3000	
3	1	1	0	0	0	2583	
4	1	0	0	1	0	6000	
5	1	1	2	1	1	5417	

```
In [23]: y.head()
```

```
Out[23]:
```

1	0
2	1
3	1
4	1
5	1

Name: Loan_Status, dtype: int64

Train TEst Split

```
In [24]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
In [25]: print(X.shape,X_train.shape,X_test.shape)

(480, 11) (384, 11) (96, 11)
```

```
In [26]: # Training the model -- Support vector machine model
classifier = svm.SVC(kernel='linear')
```

```
In [27]: classifier.fit(X_train,y_train)
```

```
Out[27]:
```

▼ SVC ⓘ ?

► Parameters

► Fitted attributes

Model Evaluation

```
In [28]: # check accuracy on training data
X_train_pred = classifier.predict(X_train)
X_train_acc = accuracy_score(y_train,X_train_pred)
print("Accuracy on Training data: ", X_train_acc)
```

Accuracy on Training data: 0.7916666666666666

```
In [30]: # check accuracy on testing data
X_test_pred = classifier.predict(X_test)
X_test_acc = accuracy_score(y_test, X_test_pred)
print("Accuracy on testing data: ", X_test_acc)
```

Accuracy on testing data: 0.8125

MModel prediction

```
In [31]: test1 = (1,1,1,1,0,4583,1508.0,128.0,360.0,1.0,0) # output: 0
test2 = (1,1,0,1,1,3000,0.0,66.0,360.0,1.0,2) # output: 1

# convert to numpy array
test1_arr = np.array(test1)
test2_arr = np.array(test2)
```

```
In [32]: print(test1_arr)
print(test2_arr)
```

```
[1.000e+00 1.000e+00 1.000e+00 1.000e+00 0.000e+00 4.583e+03 1.508e+03
 1.280e+02 3.600e+02 1.000e+00 0.000e+00]
[1.0e+00 1.0e+00 0.0e+00 1.0e+00 1.0e+00 3.0e+03 0.0e+00 6.6e+01 3.6e+02
 1.0e+00 2.0e+00]
```

```
In [33]: # reshape the array to 2d
test1_arr_reshape = test1_arr.reshape(1,-1)
test2_arr_reshape = test2_arr.reshape(1,-1)
```

```
In [34]: print(test1_arr_reshape)
print(test2_arr_reshape)
```

```
[[1.000e+00 1.000e+00 1.000e+00 1.000e+00 0.000e+00 4.583e+03 1.508e+03
 1.280e+02 3.600e+02 1.000e+00 0.000e+00]]
[[1.0e+00 1.0e+00 0.0e+00 1.0e+00 1.0e+00 3.0e+03 0.0e+00 6.6e+01 3.6e+02
 1.0e+00 2.0e+00]]
```

```
In [35]: # predict using model
pred1 = classifier.predict(test1_arr_reshape)
pred2 = classifier.predict(test2_arr_reshape)
print(pred1)
print(pred2)
```

```
[1]
[1]
```

```
D:\my project\codeit_datascience\data_science\venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
D:\my project\codeit_datascience\data_science\venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
```

```
In [38]: if pred1[0]==0:
          print("No loan")
        else:
          print("Yes loan")
```

Yes loan

```
In [39]: if pred2[0]==0:
          print("No loan")
        else:
          print("Yes loan")
```

Yes loan

```
In [ ]:
```